

Queue Management

Today Agenda

- Queue Management
 - Characteristics
 - Read operations
 - Write operations
 - Queue Implementation
- Scheduling Analysis
- Multi-source sender
- Mailbox Queue

Required Reading

- **FreeRTOS**
 - *Intoduction to FreeRTOS.pdf: chapter 4 Queue Management*
 - *The FreeRTOS™ Reference Manual: chapter 3*

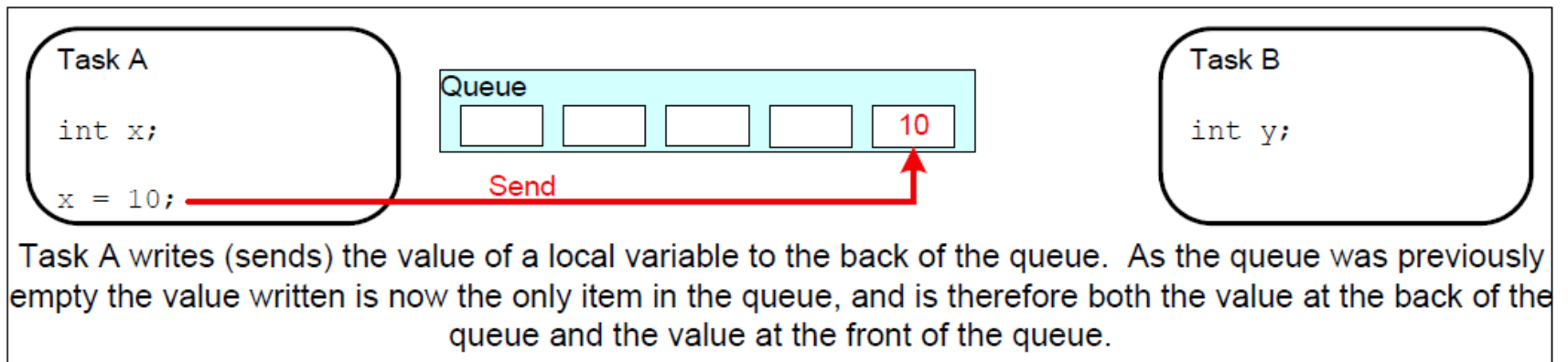
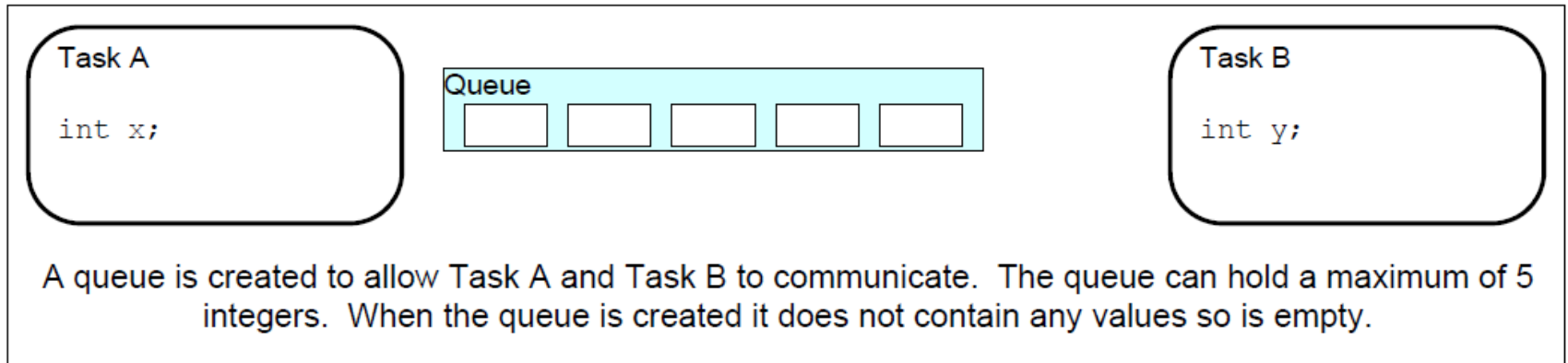
FreeRTOS Queue Characteristics

Queues provide a task-to-task, task-to-interrupt, and interrupt-to-task communication mechanism.

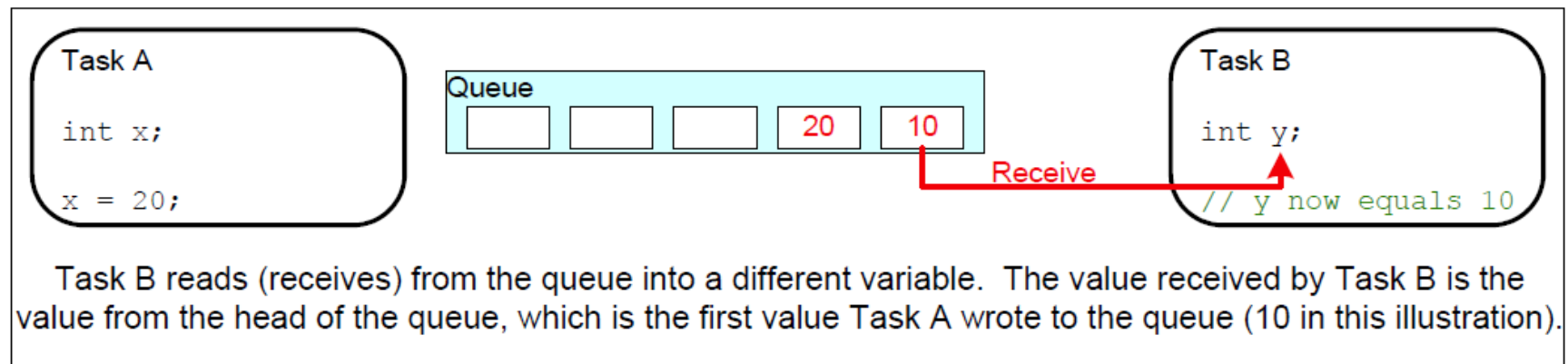
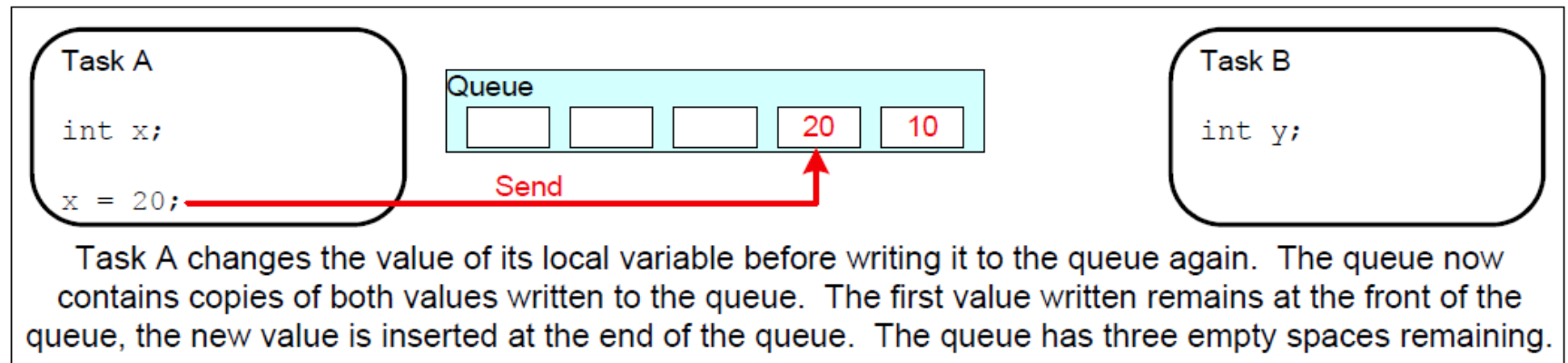
A queue can hold a finite number of fixed size data items. The maximum number of items a queue can hold is called its length. Both the length and the size of each data item are set when the queue is created.

Queues are normally used as First In First Out (FIFO) buffers, where data is written to the end (tail) of the queue and removed from the front (head) of the queue.

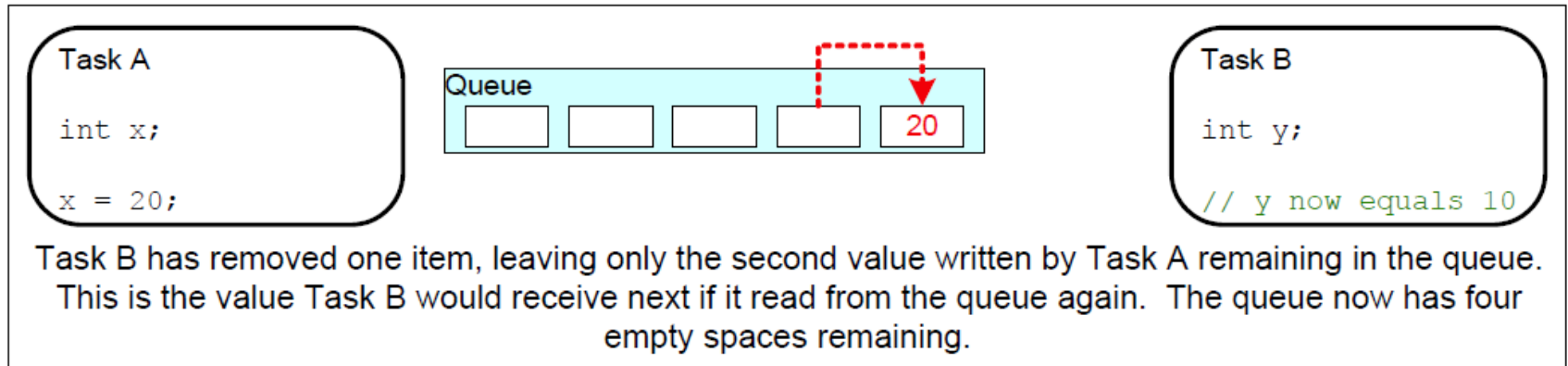
FreeRTOS Queue Characteristics



FreeRTOS Queue Characteristics



FreeRTOS Queue Characteristics

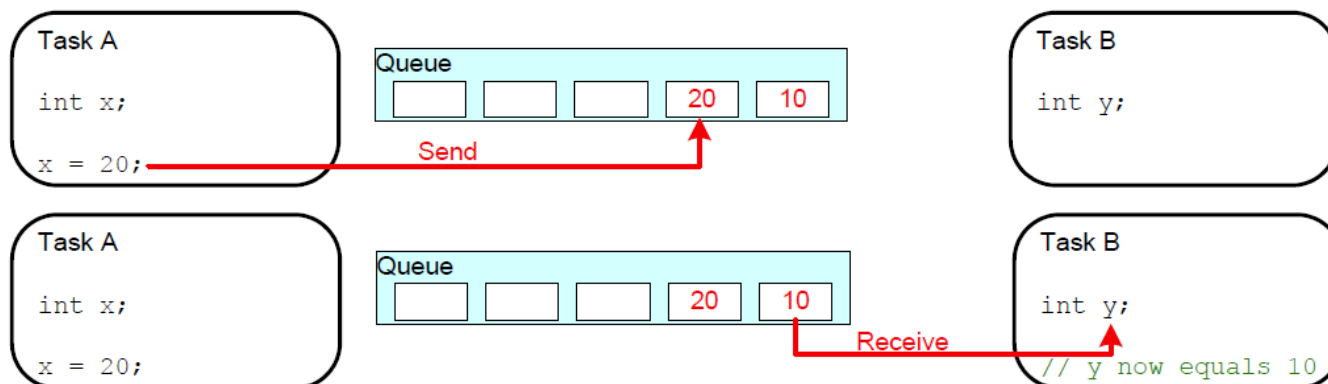


This demonstrates data being written to and read from a queue that is being used as a FIFO. It is also possible to write to the front of a queue and to overwrite data that is already at the front of a queue.

FreeRTOS Queue Characteristics

Queues are objects in their own right that can be accessed by any task that knows of their existence. Any number of tasks can write to the same queue, and any number of tasks can read from the same queue.

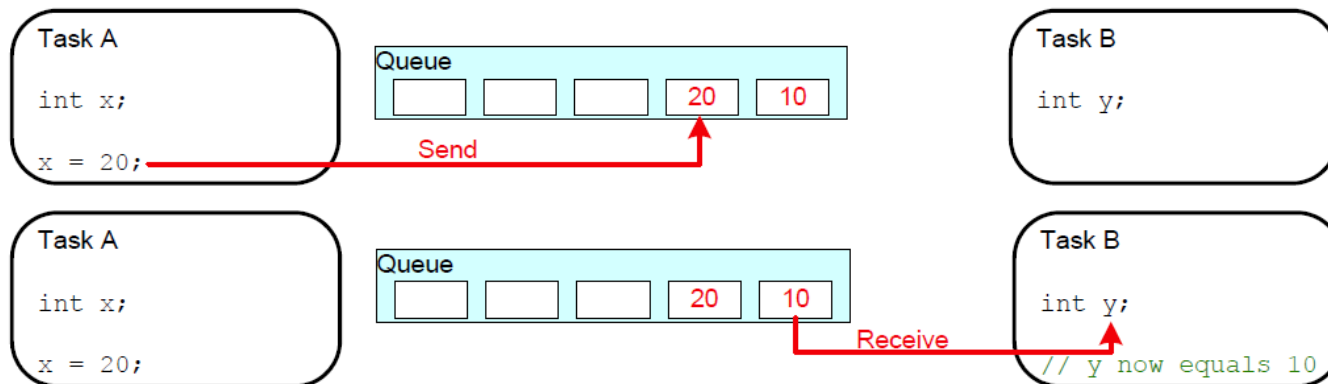
In practice it is very common for a queue to have multiple writers, but much less common for a queue to have multiple readers.



FreeRTOS Read Queue

Characteristics: block time

When a task attempts to read from a queue, it can optionally specify a block time. This is the time the task will be kept in the Blocked state to wait for data to be available from the queue should the queue already be empty.

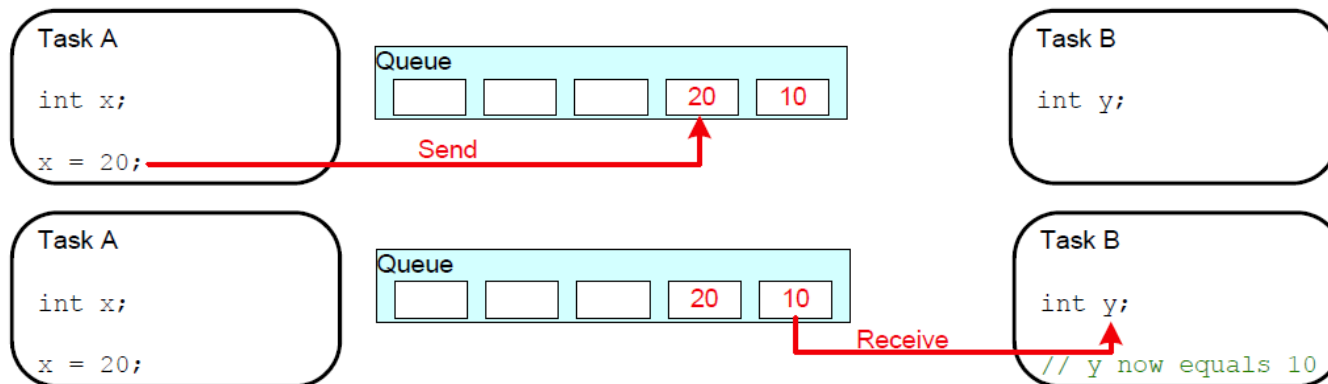


FreeRTOS Read Queue

Characteristics: block time

A task that is in the Blocked state, waiting for data to become available from a queue, is automatically moved to the Ready state when another task places data into the queue.

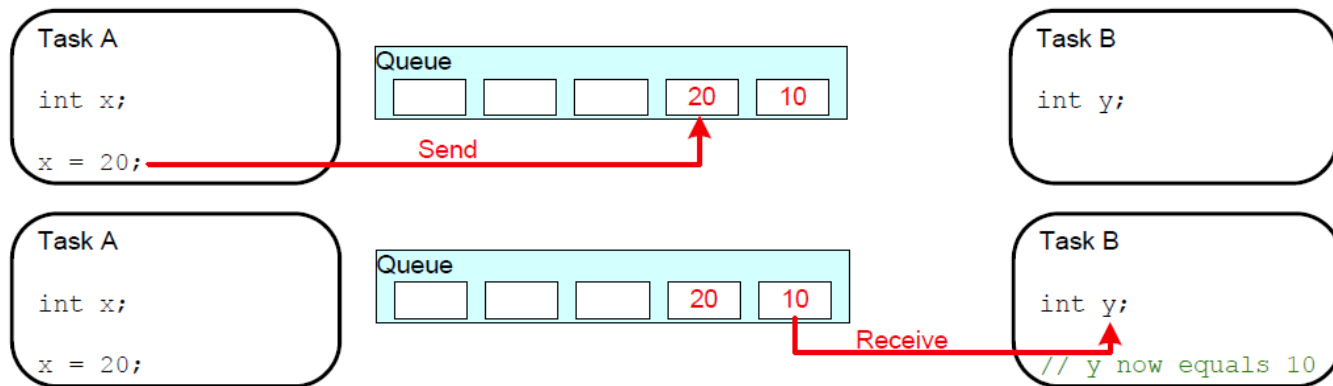
The task will also be moved automatically from the Blocked state to the Ready state if the specified block time expires before data becomes available.



FreeRTOS Read Queue

Characteristics: multi-readers

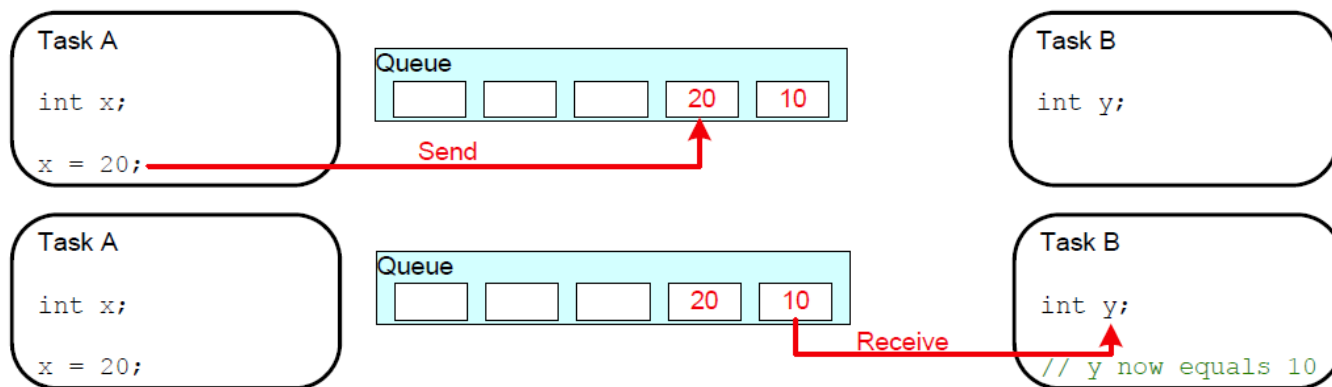
Queues can have multiple readers, so it is possible for a single queue to have more than one task blocked on it waiting for data. When this is the case, only one task will be unblocked when data becomes available.



FreeRTOS Read Queue

Characteristics: multi-readers

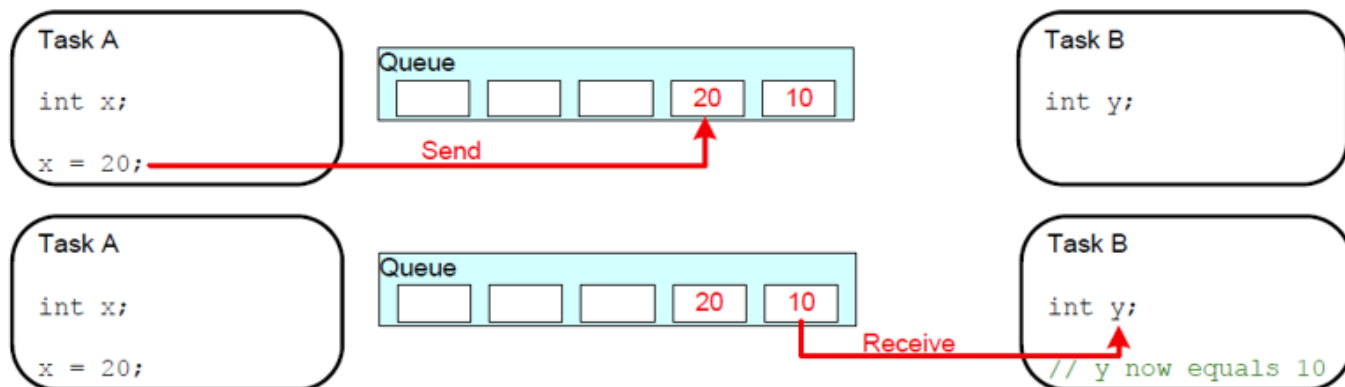
The task that is unblocked will always be the highest priority task that is waiting for data. If the blocked tasks have equal priority, then the task that has been waiting for data the longest will be unblocked.



FreeRTOS Write Queue

Characteristics: block state

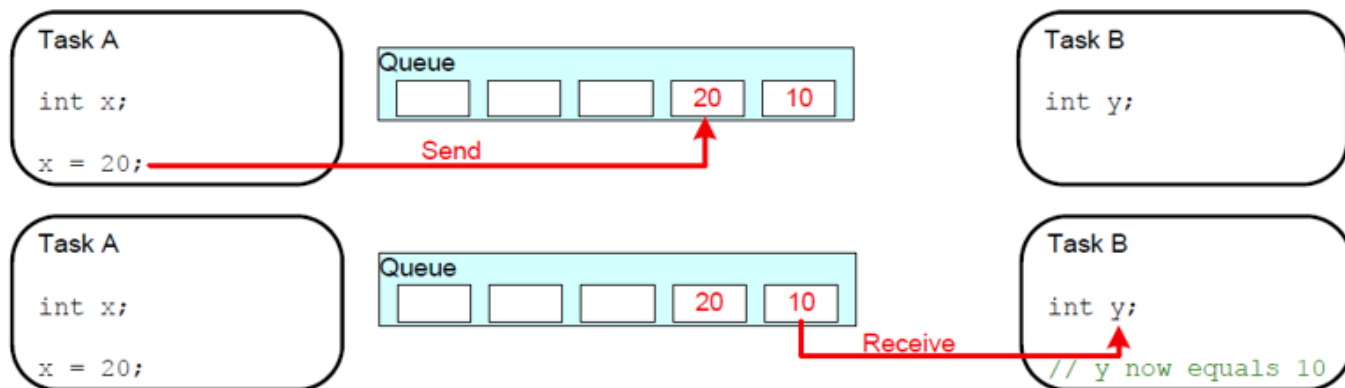
A task can optionally specify a block time when writing to a queue. In this case, the block time is the maximum time the task should be held in the Blocked state to wait for space to become available on the queue, should the queue already be full.



FreeRTOS Write Queue

Characteristics: multi-writers

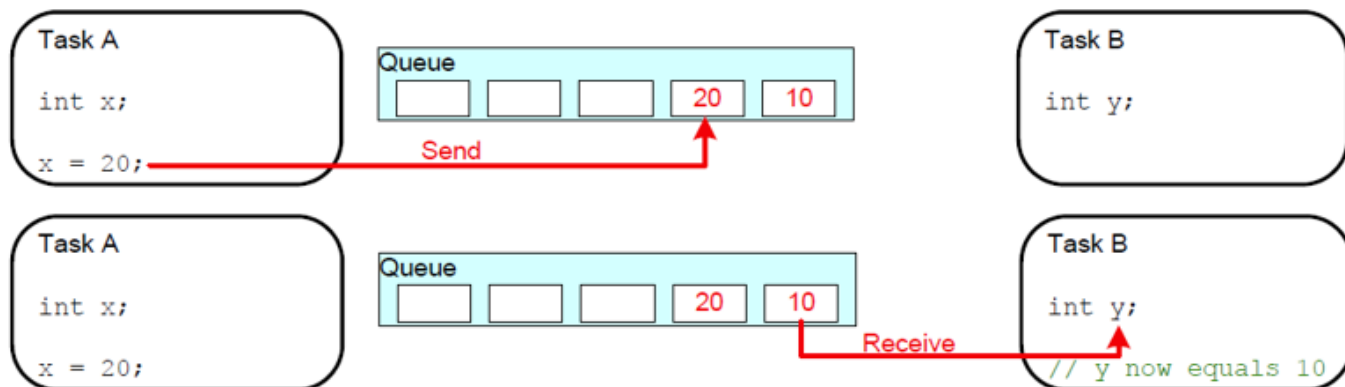
Queues can have multiple writers, so it is possible for a full queue to have more than one task blocked on it waiting to complete a send operation. When this is the case, only one task will be unblocked when space on the queue becomes available.



FreeRTOS Write Queue

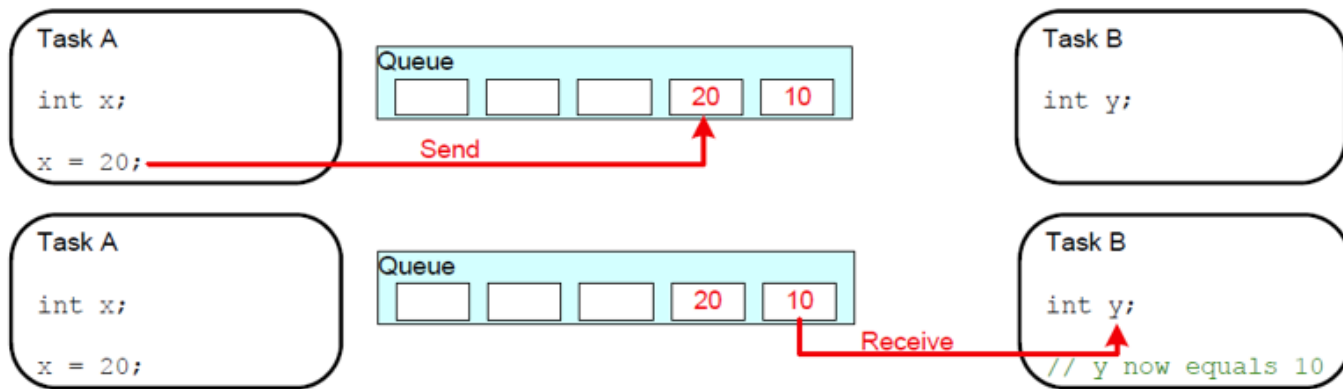
Characteristics: multi-writers

The task that is unblocked will always be the highest priority task that is waiting for space. If the blocked tasks have equal priority, then the task that has been waiting for space the longest will be unblocked.



FreeRTOS Queue Implementation

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength, UBaseType_t uxItemSize );
```



FreeRTOS Queue Implementation: write operation

xQueueSendToBack() is used to send data to the back (*tail*) of a queue and *xQueueSendToFront()* is used to send data to the front (*head*) of a queue.

```
BaseType_t xQueueSendToBack( QueueHandle_t xQueue,  
                             const void * pvItemToQueue,  
                             TickType_t xTicksToWait );
```

```
BaseType_t xQueueSendToFront( QueueHandle_t xQueue,  
                              const void * pvItemToQueue,  
                              TickType_t xTicksToWait );
```

xQueueSend() is equivalent to, and exactly the same as,
xQueueSendToBack().

FreeRTOS Queue Implementation: write operation

There are two possible return values:

pdPASS is returned only if data was successfully sent to the queue. If a block time was specified (xTicksToWait was not zero), then it is possible the calling task was placed into the Blocked state, to wait for space to become available in the queue, before the function returned.

errQUEUE_FULL is returned if data could not be written to the queue because the queue was already full.

FreeRTOS Queue Use: read operation

`xQueueReceive()` is used to receive (read) an item from a queue. The item that is received is removed from the queue.

```
BaseType_t xQueueReceive( QueueHandle_t xQueue,  
                           void * const pvBuffer,  
                           TickType_t xTicksToWait );
```

FreeRTOS Queue Use: read operation

There are two possible return values:

- **pdPASS** is returned only if data was successfully read from the queue. If a block time was specified (`xTicksToWait` was not zero), then it is possible the calling task was placed into the Blocked state to wait for data to become available on the queue. But data was successfully read from the queue before the block time expired.
- **errQUEUE_EMPTY** is returned if data cannot be read from the queue because the queue is already empty. If a block time was specified (`xTicksToWait` was not zero,) then the calling task is placed into the Blocked state to wait for another task to send data to the queue.

FreeRTOS Queue Use: status inquiry

`uxQueueMessagesWaiting()` is used to query the number of items that are currently in a queue.

Returned value is the number of items that the queue being queried is currently holding. If zero is returned, then the queue is

```
( UBaseType_t uxQueueMessagesWaiting( QueueHandle_t xQueue ) ;
```

FreeRTOS Queue Use Example

Two senders with P1 priority.
One receiver with P2 priority.
The receiving task specifies a
block time of 100 mills.
Analyze the execution orders?

```
/* Declare a variable of type QueueHandle_t. This is used to store the handle
to the queue that is accessed by all three tasks. */
QueueHandle_t xQueue;

int main( void )
{
    /* The queue is created to hold a maximum of 5 values, each of which is
    large enough to hold a variable of type int32_t. */
    xQueue = xQueueCreate( 5, sizeof( int32_t ) );

    if( xQueue != NULL )
    {
        /* Create two instances of the task that will send to the queue. The task
        parameter is used to pass the value that the task will write to the queue,
        so one task will continuously write 100 to the queue while the other task
        will continuously write 200 to the queue. Both tasks are created at
        priority 1. */
        xTaskCreate( vSenderTask, "Sender1", 1000, ( void * ) 100, 1, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, ( void * ) 200, 1, NULL );

        /* Create the task that will read from the queue. The task is created with
        priority 2, so above the priority of the sender tasks. */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

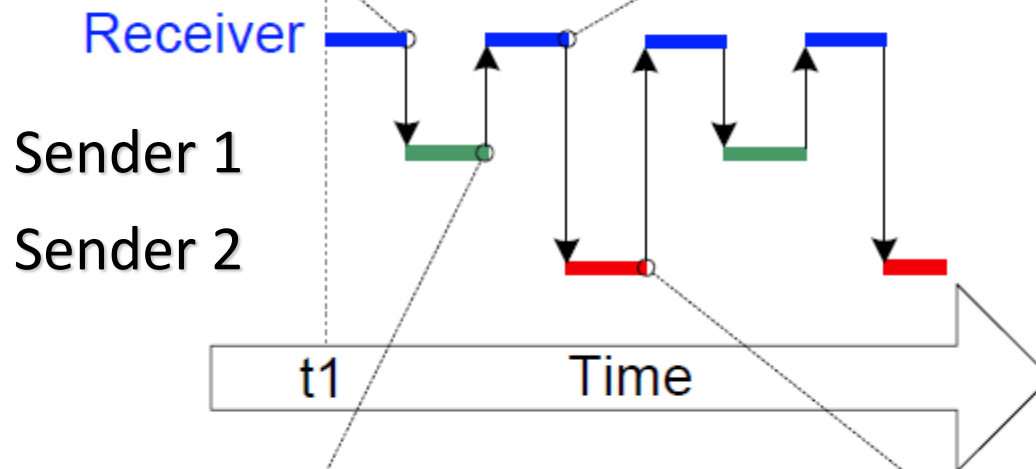
        /* Start the scheduler so the created tasks start executing. */
        vTaskStartScheduler();
    }
    else
    {
        /* The queue could not be created. */
    }

    /* If all is well then main() will never reach here as the scheduler will
    now be running the tasks. If main() does reach here then it is likely that
    there was insufficient FreeRTOS heap memory available for the idle task to be
    created. Chapter 2 provides more information on heap memory management. */
    for( ;; );
}
```

FreeRTOS Queue Use Example

1 - The Receiver task runs first because it has the highest priority. It attempts to read from the queue. The queue is empty so the Receiver enters the Blocked state to wait for data to become available. Sender 2 runs after the Receiver has blocked.

3 - The Receiver task empties the queue then enters the Blocked state again. This time Sender 1 runs after the Receiver has blocked.



2 - Sender 2 writes to the queue, causing the Receiver to exit the Blocked state. The Receiver has the highest priority so pre-empts Sender 2.

4 - Sender 1 writes to the queue, causing the Receiver to exit the Blocked state and pre-empt Sender 1 - and so it goes on

Receiver Task Implementation

```
static void vReceiverTask( void *pvParameters )
{
    /* Declare the variable that will hold the values received from the queue. */
    int32_t lReceivedValue;
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100 );

    /* This task is also defined within an infinite loop. */
    for( ;; )
    {
        /* This call should always find the queue empty because this task will
        immediately remove any data that is written to the queue. */
        if( uxQueueMessagesWaiting( xQueue ) != 0 )
        {
            vPrintString( "Queue should have been empty!\r\n" );
        }

        /* Receive data from the queue.

        The first parameter is the queue from which data is to be received. The
        queue is created before the scheduler is started, and therefore before this
        task runs for the first time.

        The second parameter is the buffer into which the received data will be
        placed. In this case the buffer is simply the address of a variable that
        has the required size to hold the received data.

        The last parameter is the block time - the maximum amount of time that the
        task will remain in the Blocked state to wait for data to be available
        should the queue already be empty. */
        xStatus = xQueueReceive( xQueue, &lReceivedValue, xTicksToWait );

        if( xStatus == pdPASS )
        {
            /* Data was successfully received from the queue, print out the received
            value. */
            vPrintStringAndNumber( "Received = ", lReceivedValue );
        }
        else
        {
            /* Data was not received from the queue even after waiting for 100ms.
            This must be an error as the sending tasks are free running and will be
            continuously writing to the queue. */
            vPrintString( "Could not receive from the queue.\r\n" );
        }
    }
}
```


Example: Queue Scheduling, Fixed Priority

Five FreeRTOS tasks (T) are created with initial priorities (P): T1-P1, T2-P2, T3-P2, T4-P3 and T5-P4 and a single queue Q of 5 elements. An Idle task is created by default. The tick interval is 5 msec (not the default 10 msec) and time slice scheduling is used and. Scheduling starts at 0 msec. All send and receive queue functions for the tasks have an indefinite wait time.

Example: Queue Scheduling, Fixed Priority

The task functionalities are as follows:

- a) T1-P1 when running for 5 msec sends one element to Q at the end of its task
- b) T2-P2 when running for 10 msec receives two elements from Q at the end of its task
- c) T3-P2 when running for 5 msec receives one element from Q at the end of its task
- d) T4-P3 when running for 10 msec sends one element to Q at the end of its task
- e) T5-P4 sets the delay parameter of `vTaskDelayUntil` for 45 msec at the beginning of its task and runs for 5 msec

Example: Queue Scheduling, Fixed Priority

Queue size: 1 2 3 4 5 4 5 3

Task 5	xxx									xxx									
Task 4		xxx	xxx	xxx	xxx	xxx	xxx	xxx	xxx		xxx	xxx			xxx	xxx		xxx	
Task 3														xxx					
Task 2													xxx				xxx		
Task 1																			
Idle Task																			
	0	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90

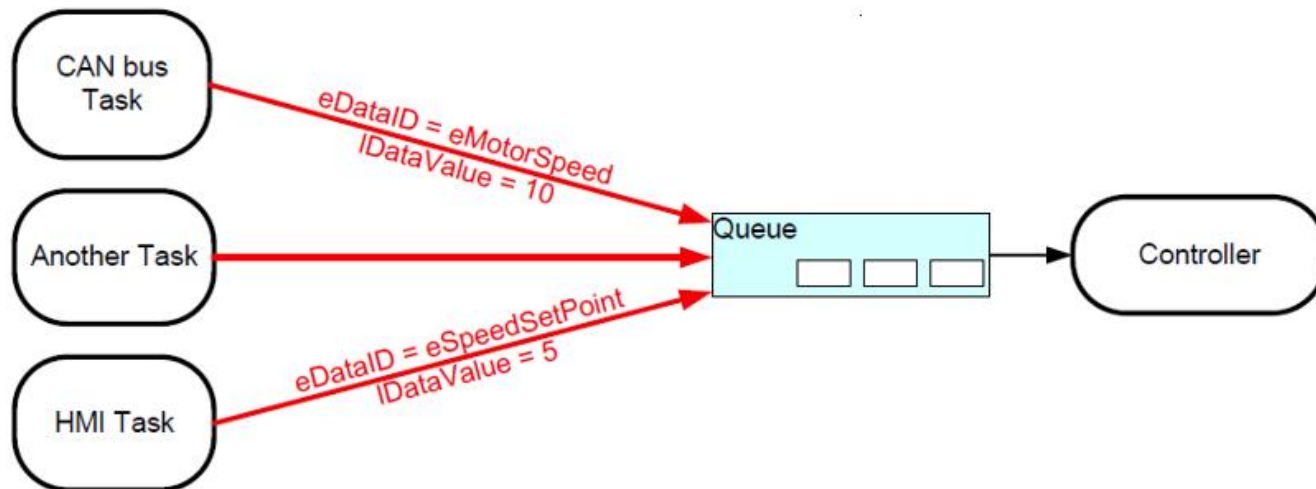
Queue size: 4 5 4 5 4 5 3 4 5 4

Task 5	xxx									xxx									
Task 4		xxx	xxx	xxx		xxx	xxx				xxx	xxx		xxx	xxx	xxx	xxx		
Task 3					xxx				xxx										xxx
Task 2								xxx					xxx						
Task 1																			
Idle Task																			

9095100105110115120125130135140145150155160165170175180

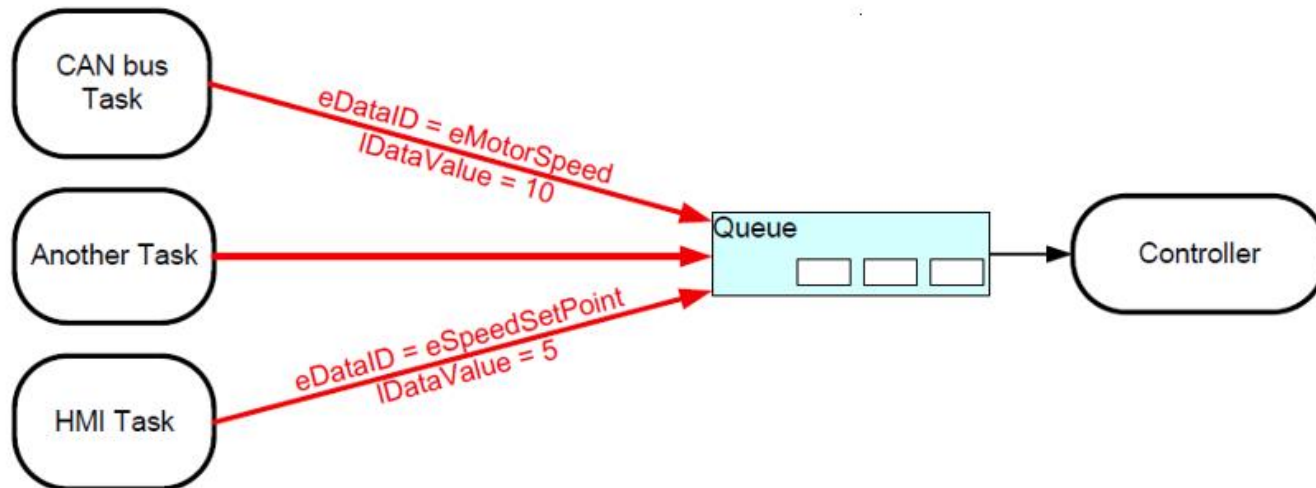
FreeRTOS Data from Multiple Sources

FreeRTOS designs for a task commonly receive data from more than one source. The receiving task needs to know where the data came from to determine how the data should be processed.



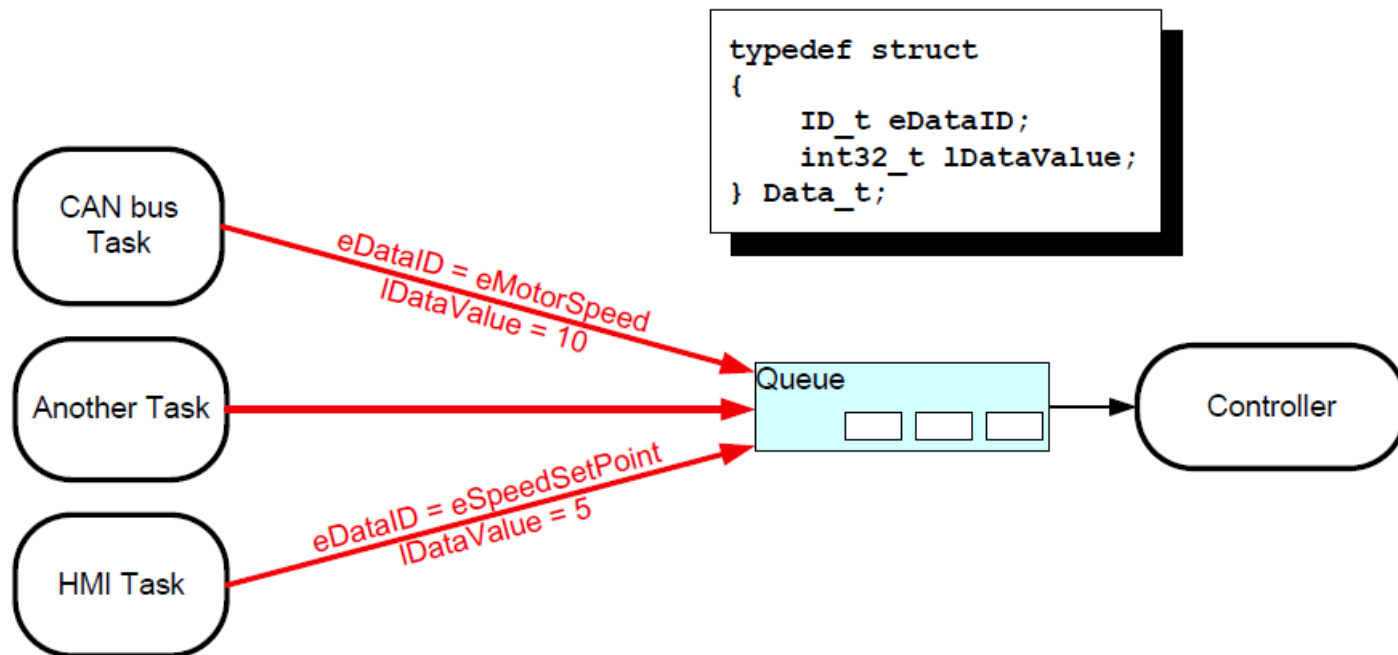
FreeRTOS Data from Multiple Sources

A solution is to use a single queue to transfer structures with both the value of the data and the source of the data contained in the structure's fields.



FreeRTOS Data from Multiple Sources

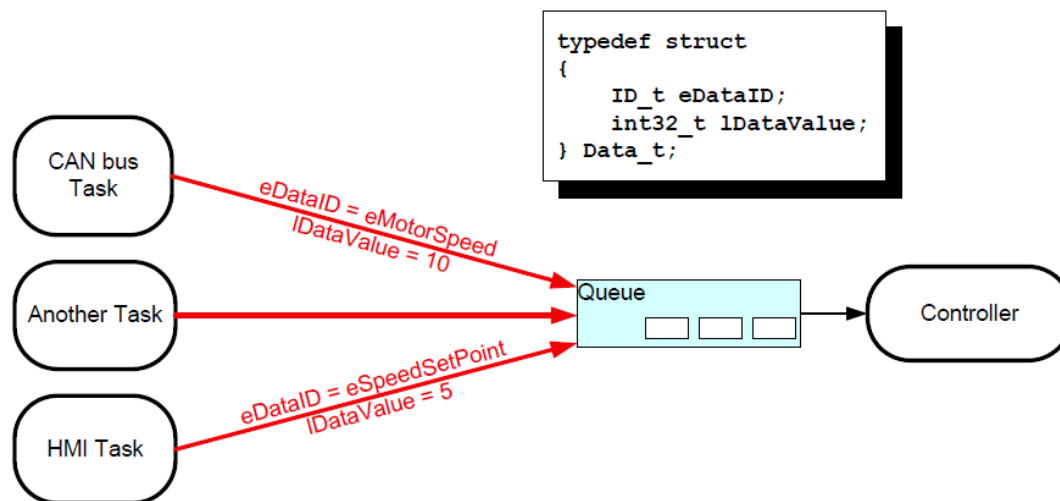
A queue is created that holds structures of type `Data_t`. The structure members allow both a data value and an enumerated type indicating what the data means to be sent to the queue in one message.



FreeRTOS Data from Multiple Sources

The next example is similar to the previous example, but the task priorities are reversed, so the receiving task has a lower priority than the sending tasks. Also, the queue is used to pass structures, rather than integers.

The example uses a structure that is to be passed on a queue and a declaration of two variables.



FreeRTOS Data from Multiple Sources

```
int main( void )
{
    /* The queue is created to hold a maximum of 3 structures of type Data_t. */
    xQueue = xQueueCreate( 3, sizeof( Data_t ) );

    if( xQueue != NULL )
    {
        /* Create two instances of the task that will write to the queue. The
        parameter is used to pass the structure that the task will write to the
        queue, so one task will continuously send xStructsToSend[ 0 ] to the queue
        while the other task will continuously send xStructsToSend[ 1 ]. Both
        tasks are created at priority 2, which is above the priority of the receiver. */
        xTaskCreate( vSenderTask, "Sender1", 1000, &(amp; xStructsToSend[ 0 ] ), 2, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, &(amp; xStructsToSend[ 1 ] ), 2, NULL );

        /* Create the task that will read from the queue. The task is created with
        priority 1, so below the priority of the sender tasks. */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, NULL );

        /* Start the scheduler so the created tasks start executing. */
        vTaskStartScheduler();
    }
    else
    {
        /* The queue could not be created. */
    }

    /* If all is well then main() will never reach here as the scheduler will
    now be running the tasks. If main() does reach here then it is likely that
    there was insufficient heap memory available for the idle task to be created.
    Chapter 2 provides more information on heap memory management. */
    for( ;; );
}
```


FreeRTOS Mailboxes

There is no consensus on terminology within the embedded community, and mailbox will mean different things in different RTOSes.

In FreeRTOS the term mailbox is used to refer to a queue that has a length of one. A queue may be described as a mailbox because of the way it is used in the application, rather than because it has a functional difference to a queue.



FreeRTOS Mailboxes

A queue is used to send data from one task to another task. The sender places an item in the queue, and the receiver removes the item from the queue. The data passes through the queue from the sender to the receiver.



FreeRTOS Mailboxes

A mailbox is used to hold data that can be read by any task. The data does not pass through the mailbox, but instead remains in the mailbox until it is overwritten.

The sender overwrites the value in the mailbox. The receiver reads the value from the mailbox but does not remove the value from the mailbox.



FreeRTOS Mailboxes

This example is a queue as a mailbox.

```
/* A mailbox can hold a fixed size data item. The size of the data item is set
when the mailbox (queue) is created. In this example the mailbox is created to
hold an Example_t structure. Example_t includes a time stamp to allow the data held
in the mailbox to note the time at which the mailbox was last updated. The time
stamp used in this example is for demonstration purposes only - a mailbox can hold
any data the application writer wants, and the data does not need to include a time
stamp. */
typedef struct xExampleStructure
{
    TickType_t xTimeStamp;
    uint32_t ulValue;
} Example_t;

/* A mailbox is a queue, so its handle is stored in a variable of type
QueueHandle_t. */
QueueHandle_t xMailbox;

void vAFunction( void )
{
    /* Create the queue that is going to be used as a mailbox. The queue has a
length of 1 to allow it to be used with the xQueueOverwrite() API function, which
is described below. */
    xMailbox = xQueueCreate( 1, sizeof( Example_t ) );
}
```

FreeRTOS Mailboxes

Like the `xQueueSendToBack()` API function, the `xQueueOverwrite()` API function sends data to a queue.

```
BaseType_t xQueueOverwrite( QueueHandle_t xQueue, const void * pvItemToQueue );
```

But unlike `xQueueSendToBack()`, if the queue is already full, then `xQueueOverwrite()` will overwrite data that is already in the queue.



FreeRTOS Mailboxes

`xQueueOverwrite()` should only be used with queues that have a length of one. That restriction avoids the need for the function's implementation to make an arbitrary decision as to which item in the queue to overwrite, if the queue is full.

```
BaseType_t xQueueOverwrite( QueueHandle_t xQueue, const void * pvItemToQueue );
```



FreeRTOS Mailboxes

xQueue is the handle of the queue to which the data is being sent (written). The queue handle will have been returned from the call to xQueueCreate() used to create the queue.

pvItemToQueue is a pointer to the data to be copied into the queue. The size of each item that the queue can hold is set when the queue is created.

Returned value xQueueOverwrite() writes to the queue even when the queue is full, so pdPASS is the only possible return value.

```
BaseType_t xQueueOverwrite( QueueHandle_t xQueue, const void * pvItemToQueue );
```

FreeRTOS Mailboxes

In this example xQueueOverwrite() being used to write to the mailbox (queue) that was created previously.

```
void vUpdateMailbox( uint32_t ulNewValue )
{
    /* Example_t was defined in Listing 67. */
    Example_t xData;

    /* Write the new data into the Example_t structure.*/
    xData.ulValue = ulNewValue;

    /* Use the RTOS tick count as the time stamp stored in the Example_t structure. */
    xData.xTimeStamp = xTaskGetTickCount();

    /* Send the structure to the mailbox - overwriting any data that is already in the
    mailbox. */
    xQueueOverwrite( xMailbox, &xData );
}
```


FreeRTOS Mailboxes

`xQueuePeek()` is used to receive (read) an item from a queue without the item being removed from the queue. `xQueuePeek()` receives data from the head of the queue, without modifying the data stored in the queue, or the order in which data is stored in the queue.

`xQueuePeek()` has the same function parameters and return value as `xQueueReceive()`.

The next example shows `xQueuePeek()` being used to receive the item posted to the mailbox (queue) in the previous example.

FreeRTOS Mailboxes

```
BaseType_t vReadMailbox( Example_t *pxData )
{
    TickType_t xPreviousTimeStamp;
    BaseType_t xDataUpdated;

    /* This function updates an Example_t structure with the latest value received
    from the mailbox. Record the time stamp already contained in *pxData before it
    gets overwritten by the new data. */
    xPreviousTimeStamp = pxData->xTimeStamp;

    /* Update the Example_t structure pointed to by pxData with the data contained in
    the mailbox. If xQueueReceive() was used here then the mailbox would be left
    empty, and the data could not then be read by any other tasks. Using
    xQueuePeek() instead of xQueueReceive() ensures the data remains in the mailbox.
    A block time is specified, so the calling task will be placed in the Blocked
    state to wait for the mailbox to contain data should the mailbox be empty. An
    infinite block time is used, so it is not necessary to check the value returned
    from xQueuePeek(), as xQueuePeek() will only return when data is available. */
    xQueuePeek( xMailbox, pxData, portMAX_DELAY );

    /* Return pdTRUE if the value read from the mailbox has been updated since this
    function was last called. Otherwise return pdFALSE. */
    if( pxData->xTimeStamp > xPreviousTimeStamp )
    {
        xDataUpdated = pdTRUE;
    }
    else
    {
        xDataUpdated = pdFALSE;
    }

    return xDataUpdated;
}
```